Unit2 - Python Foundation

Variables
- Definition
  - Name used to store data
- Basic Data Types
  - str
  - int
  - float
  - bool
- Rules
  - Naming rules
    - Don't start with a number
    - Don't use hyphen (-)
    - Don't use reserved keywords like class, return, for, try, pass, etc.
    - Start with a letter or underscore (_)
    - Case-sensitive: age != Age

Operations and Syntax
- Code structure
  - Uses clean indentation
  - No semicolons or curly braces needed
  - Uses # for comments
- Python operations
  - Arithmetic: +, -, **
  - Comparison: ==, >
  - Logical: and, or, not

Logic Statements
- Comparison Operators
  - Used to compare two values
  - Return True or False
  - Operators and Meaning
    - == equal to
    - != not equal to
    - > greater than
    - < less than
    - >= greater than or equal to
    - <= less than or equal to
- Logical Operators
  - Combine multiple conditions
  - Operators and Meaning
    - and: True only if both conditions true
    - or: True if at least one condition true
    - not: reverses a boolean value

Loops and Iterations
- Types of Loops
  - For Loop
    - Best for known sequence or range like list, string, range, dict
    - Controlled by iterating over items
  - While Loop
    - Repeats until condition becomes false
    - Controlled by condition check
- Loop Control Statements
  - Statements and Effects
    - Break: Immediately exits nearest enclosing loop
    - Continue: Skips rest of current iteration and jumps back to condition/next item
    - Pass: Does nothing, placeholder when syntax requires a statement
- Note!
  - Every while loop needs an exit strategy

Functions
- Purpose
  - Reuse code
  - Reduce repetition
  - Organize code
  - Easier maintenance
- Components
  - Defining functions
    - def function_name():
    - def function_name(parameters):
  - Calling functions
    - function_name()
    - function_name(arguments)
  - Return statement
    - return value
- Examples
  - Function with no parameter: def hello():
  - Function with parameter: def hello(name):
  - Function with no return: prints inside function
  - Function with return: returns value
- Built-in Functions
  - len()
  - sum()
  - max()
  - min()
  - type()

Collections
- Types and Characteristics
  - List []
    - Ordered
    - Mutable
    - Allows duplicates
  - Tuple ()
    - Ordered
    - Immutable
    - Allows duplicates
  - Set {}
    - Unordered
    - Mutable
    - Unique elements
    - Good for removing duplicates
  - Dictionary {}
    - Key:Value pairs
    - Unordered
    - Mutable
    - Fast access by key

Comprehensions and Copying
- Comprehensions
  - List: [expression for x in iterable if condition]
  - Set: {expression for x in iterable if condition}
  - Dictionary: {key: value for x in iterable if condition}
  - Generator: (expression for x in iterable if condition)
- Copying
  - b = a: same object, changes affect both
  - b = a.copy(): shallow copy, new outer object, nested mutable shared
  - b = deepcopy(a): deep copy, completely independent nested objects

Classes
- Definition
  - Blueprint for creating objects
  - class Student: pass
- Object
  - An object is an instance of a class
  - student1 = Student()
- __init__
  - Called automatically when creating new object
  - def __init__(self, name):
  - self.name = name
- Attributes
  - Variables storing object information
  - Examples: self.name, self.age
- Methods
  - Functions inside class
  - Example: def display_info(self): print(self.name)
- self keyword
  - Refers to current object

File Operations and Handling
- Modes
  - "r": read existing file
  - "w": write and replace content
  - "a": append after existing content
  - "x": create only if absent
- Methods
  - read()
  - read_text()
  - write()
  - existst
  - is_file()
  - is_dir()

Presented with xmind